

Composition de composants avec slots et templates

Objectifs de cette section

Vous avez appris à créer un composant Web simple. Dans cette section, vous allez découvrir comment rendre vos composants plus **flexibles et modulaires**, grâce aux **slots** et aux **templates HTML**.

À la fin de cette section, vous serez capable de :

- Utiliser un `<slot>` pour permettre l'injection de contenu dans un composant.
- Créer un composant générique réutilisable avec différents contenus.
- Utiliser `<template>` pour définir des structures HTML clonables.
- Structurer des composants comme des "boîtes à contenu".

1. Le slot : un espace réservé au contenu externe

Un composant peut définir un `<slot>` à l'intérieur de son HTML. Ce slot peut être alimenté par l'utilisateur du composant.

- Si aucun contenu n'est fourni, on peut définir un **contenu par défaut** dans le slot.
- Un composant peut avoir **plusieurs slots nommés** pour plus de flexibilité.

Exemple : composant avec slot simple

```
<!-- index.html -->
<slot-component>
  <p>Contenu injecté dans le composant.</p>
</slot-component>

<script type="module" src="slot-component.js"></script>
```

```
// slot-component.js
class SlotComponent extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({mode: "open"});
    this.shadowRoot.innerHTML = `
    <div style="border: 1px solid #ccc; padding: 10px;">
      <slot></slot>
    </div>
    `;
  }
}

customElements.define("slot-component", SlotComponent);
```

2. Slots nommés

Les **slots nommés** permettent de mieux contrôler la structure du composant. L'utilisateur du composant peut injecter du contenu à des endroits spécifiques.

- On utilise `<slot name="...">` dans le composant.
- On utilise l'attribut `slot="..."` dans l'appel.

Exemple : slot nommé pour titre et contenu

```
<!-- index.html -->
<named-slot>
  <span slot="title">Ma tâche</span>
  <p slot="content">Contenu personnalisé de la tâche.</p>
</named-slot>

<script type="module" src="named-slot.js"></script>
```

```
// named-slot.js
class NamedSlot extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({mode: "open"});
    this.shadowRoot.innerHTML = `
    <div>
      <h3><slot name="title">Titre par défaut</slot></h3>
      <div><slot name="content">Contenu par défaut</slot></div>
    </div>
    `;
  }
}

customElements.define("named-slot", NamedSlot);
```

3. Utiliser `<template>` pour générer du contenu

Un `<template>` HTML permet de définir une structure HTML qui ne sera pas affichée directement, mais qui pourra être clonée via JavaScript.

- Cela permet de créer des composants **légers, structurés et réutilisables**.
- Combiné avec le Shadow DOM, cela renforce l'encapsulation.

Exemple : clonage d'un template

```
<!-- index.html -->
<template id="task-template">
  <li class="task">
    <span class="label"></span>
  </li>
</template>

<ul id="task-list"></ul>
```

```
// script.js
const template = document.getElementById("task-template");
const clone = template.content.cloneNode(true);
clone.querySelector(".label").textContent = "Tâche clonée depuis un template";
document.getElementById("task-list").appendChild(clone);
```

4. Exercice de mise en pratique

Consignes

1. Créez un composant `<task-box>` qui :
 - Utilise un `<slot>` pour insérer dynamiquement le contenu d'une tâche.
 - Affiche une structure générique (par exemple une boîte grise ou une carte).
 - Peut contenir du texte, un bouton ou n'importe quel autre élément.
2. Créez une page HTML avec **au moins trois** instances de `<task-box>`, contenant chacune un contenu différent (texte seul, texte + bouton, texte + lien...).

Organisation attendue

```
/exercices
├── 08-task-box
│   ├── index.html
│   └── task-box.js
```